

EXAM RECALL SOLUTIONS

SOURCE: Exam Recall Notes

FORMAT: 3 Coding Questions

STACK: Python 3

How to read this guide: Each question is laid out the way it appeared on screen — problem statement, input format, constraints, sample cases, followed by an approach, a working Python 3 solution, and complexity analysis.

Q.01 Minimum Vote Flips for Candidate 1

GREEDY / SORTING

The Problem

You have N voters, each with a vote (0 or 1) and a weight $w[i]$. You may flip up to K votes (change $0 \rightarrow 1$ or $1 \rightarrow 0$).

After flipping, Candidate 1 wins if the total weight of votes for 1 is strictly greater than the total weight of votes for 0.

Determine whether Candidate 1 can win, and if so, return the minimum number of flips required. Return -1 if Candidate 1 cannot win even with K flips.

Input Format & Constraints

- **Line 1:** Integer N — number of votes.
- **Line 2:** Integer K — maximum flips allowed.
- **Next N lines:** Integer $v[i]$ — the vote (0 or 1).
- **Next N lines:** Integer $w[i]$ — the weight.
- **Constraints:** $1 \leq N \leq 10^5$, $1 \leq K \leq 10^5$, $v[i] \in \{0, 1\}$, $0 \leq w[i] \leq 10^9$

Sample Cases

CASE 01

OUTPUT: 0

Input:

 $N=3, K=1$ $v = [1, 0, 1]$ $w = [10, 5, 10]$

Details:

 $S_1 = 20, S_0 = 5$.

Candidate 1 is already winning. 0 flips needed.

CASE 02

OUTPUT: 1

Input:

 $N=4, K=2$ $v = [0, 0, 0, 1]$ $w = [10, 20, 5, 5]$

Details:

 $S_1 = 5, S_0 = 35$. Flip heaviest 0-voter ($w=20$).

Now $S_1 = 25, S_0 = 15$. Candidate 1 wins in 1 flip.

CASE 03

OUTPUT: -1

Input:

 $N=3, K=1$ $v = [0, 0, 0]$ $w = [100, 100, 100]$

Details:

$S_1 = 0, S_0 = 300$. Even after 1 allowed flip, $S_1 = 100, S_0 = 200$. Impossible.

The Approach

Time: $O(N \log N)$ | Space: $O(N)$

1. Compute initial sums S_1 (total weight for 1) and S_0 (total weight for 0).
2. If $S_1 > S_0$, return 0 .
3. Sort all 0-voters by weight in descending order.

4. Flip the heaviest 0-voter first (each flip shifts total gap by $2 \times w$).
5. After each flip (up to K), check if $S_1 > S_0$ and return flip count if true.
6. If K flips do not suffice, return `-1`.

Python 3 Solution

```
import sys

def solve(n, k, v, w):
    s1 = 0
    s0 = 0
    zero_w = []

    for i in range(n):
        if v[i] == 1:
            s1 += w[i]
        else:
            s0 += w[i]
            zero_w.append(w[i])

    if s1 > s0:
        return 0

    zero_w.sort(reverse=True)
    flips = 0

    for weight in zero_w:
        if flips >= k:
            break
        s0 -= weight
        s1 += weight
        flips += 1
        if s1 > s0:
            return flips

    return -1

if __name__ == "__main__":
    try:
        input_data = sys.stdin.read().split()
        if input_data:
            n = int(input_data[0])
            k = int(input_data[1])
            v = [int(x) for x in input_data[2:2+n]]
            w = [int(x) for x in input_data[2+n:2+2*n]]
            print(solve(n, k, v, w))
    except (EOFError, ValueError):
        pass
```

Q.02 Tiling a Strip with Penalties

DYNAMIC PROGRAMMING

The Problem

A strip of length N must be fully tiled using tiles of width 1, 2, or 3. Placing a tile of width w starting at position i costs $c[w] + \text{penalty}[i]$, where $c[1]$, $c[2]$, $c[3]$ are fixed tile costs and $\text{penalty}[i]$ is a per-position surcharge applied at the tile's starting position.

Positions are 0-indexed (spanning 0 to $N-1$). Find the minimum total cost to tile the entire strip.

Input Format & Constraints

- **Line 1:** Integer N — length of strip.
- **Lines 2-4:** Integers $C1, C2, C3$ — cost of width 1, 2, and 3 tiles.
- **Line 5:** N space-separated integers — $penalties[i]$.
- **Constraints:** $1 \leq N \leq 10^5$, $1 \leq C1, C2, C3 \leq 10^5$, $-10^9 \leq penalties[i] \leq 10^9$

Sample Cases

CASE 01 OUTPUT: 11	CASE 02 OUTPUT: 8	CASE 03 OUTPUT: 103
Input: <code>N=3, C1=10, C2=10, C3=10 penalties = [1, 1, 1]</code>	Input: <code>N=4, C1=1, C2=100, C3=100 penalties = [1, 1, 1, 1]</code>	Input: <code>N=5, C1=100, C2=1, C3=100 penalties = [1, 100, 1, 100, 1]</code>
Details: One width-3 tile at pos 0. Cost = 10 + 1 = 11.	Details: Width-1 tiles are 100x cheaper. Four of them at cost (1+1) × 4 = 8.	Details: Width-2 at 0 (cost 2) + width-3 at 2 (cost 101) = 103. High-penalty positions skipped.

The Approach

Time: $O(N)$ | **Space:** $O(N)$

1. Let $dp[i]$ = minimum cost to tile from position i to $N-1$.
2. Base case: $dp[N] = 0$.
3. For each i from $N-1$ down to 0 , evaluate valid choices:
 $dp[i] = \min(c[w] + penalty[i] + dp[i+w])$ for $w \in \{1, 2, 3\}$ where $i+w \leq N$.
4. The result is in $dp[0]$.

Python 3 Solution

```
import sys

def solve(N, C1, C2, C3, penalties):
    dp = [0] * (N + 1)

    for i in range(N - 1, -1, -1):
        best = C1 + penalties[i] + dp[i + 1]

        if i + 2 <= N:
            best = min(best, C2 + penalties[i] + dp[i + 2])
        if i + 3 <= N:
            best = min(best, C3 + penalties[i] + dp[i + 3])

        dp[i] = best

    return dp[0]

if __name__ == "__main__":
    try:
        input_data = sys.stdin.read().split()
        if input_data:
            N = int(input_data[0])
            C1, C2, C3 = int(input_data[1]), int(input_data[2]), int(input_data[3])
            penalties = [int(x) for x in input_data[4:4+N]]
            print(solve(N, C1, C2, C3, penalties))
    except (EOFError, ValueError):
        pass
```

Q.03 Phase Scheduling under Duration Cap

BRUTE FORCE / ENUMERATION

The Problem

You have N tasks and P phases. Task i assigned to phase p contributes value $v[i][p]$ and takes duration $d[i][p]$.

Each phase runs sequentially; within a phase, tasks run in parallel. The duration of a phase equals the maximum task duration in that phase. Each task must be assigned to exactly one phase.

The total project duration is the sum of phase durations. Maximize total value subject to total project duration $\leq T$. Return -1 if no valid assignment exists.

Input Format & Constraints

- **Line 1:** Integer N — tasks, **Line 2:** Integer P — phases, **Line 3:** Integer T — max duration.
- **Next N lines:** P space-separated integers — row i of matrix v .
- **Next N lines:** P space-separated integers — row i of matrix d .
- **Constraints:** $1 \leq N \leq 8$, $1 \leq P \leq 4$, $1 \leq T \leq 100$, $1 \leq v[i][j]$, $d[i][j] \leq 20$

Sample Cases

CASE 01 OUTPUT: 19 Input: N=3, P=2, T=10 v = [[5,10],[8,2],[4,6]] d = [[4,6],[5,3],[2,5]] Details: Assign tasks 0,1 to Phase 0; task 2 to Phase 1. 1. Max durations: $\max(4,5)=5$ & $\max(5)=5$. Total = $10 \leq 10$. Value = $5+8+6 = 19$.	CASE 02 OUTPUT: -1 Input: N=2, P=2, T=2 v = [[10,10],[10,10]] d = [[5,5],[5,5]] Details: Every task duration is 5. Any assignment yields total duration $\geq 5 > 2$. Impossible.	CASE 03 OUTPUT: 127 Input: N=7, P=3, T=79 v = 7x3 matrix d = 7x3 matrix Details: Optimal phase durations: 20, 14, 14 (Sum $48 \leq 79$). Total max value achieved = 127.
---	---	--

The Approach

Time: $O(P^N \cdot N)$ | **Space:** $O(N \cdot P)$

1. Constraints are small ($N \leq 8$, $P \leq 4$). Maximum states: $4^8 = 65,536$.
2. Enumerate every assignment as a base- P integer with N digits.
3. For each state, compute per-phase max task duration and total accumulated value.
4. If sum of phase durations $\leq T$, update max value.
5. Return maximum value found, or **-1** if no valid setup exists.

Python 3 Solution

```
import sys

def solve(n, p, t, v, d):
    best = -1
    total = p ** n

    for mask in range(total):
        phase_max = [0] * p
        value = 0
        temp = mask

        for i in range(n):
            ph = temp % p
            temp //= p
            value += v[i][ph]
            if d[i][ph] > phase_max[ph]:
                phase_max[ph] = d[i][ph]

        if sum(phase_max) <= t and value > best:
            best = value

    return best

if __name__ == "__main__":
    try:
        input_data = sys.stdin.read().split()
        if input_data:
            n = int(input_data[0])
            p = int(input_data[1])
            t = int(input_data[2])
            idx = 3
            v = []
            for _ in range(n):
                v.append([int(x) for x in input_data[idx:idx+p]])
                idx += p
            d = []
            for _ in range(n):
                d.append([int(x) for x in input_data[idx:idx+p]])
                idx += p
            print(solve(n, p, t, v, d))
    except (EOFError, ValueError):
        pass
```